

Система безопасности в базах данных

Как только данные структурированы и сведены в БД, возникает проблема организации доступа к ним множества пользователей. Очевидно, что нельзя позволить всем без исключения пользователям беспрепятственный доступ ко всем элементам БД. В любой БД существуют конфиденциальные данные, доступ к которым может быть разрешен лишь ограниченному кругу лиц. Так, в банковской системе особо конфиденциальными могут считаться, например, сведения о выданных кредитах. Это только один из аспектов проблемы безопасности в СУБД.

В самом общем виде требования к безопасности реляционных СУБД можно сформулировать так:

- данные в любой таблице должны быть доступны не всем пользователям, а лишь некоторым из них;
- некоторым пользователям должно быть разрешено обновлять данные в таблицах, в то время как для других допускается лишь выбор данных из этих же таблиц;
- для некоторых таблиц необходимо обеспечить выборочный доступ к ее столбцам;
- некоторым пользователям должен быть запрещен непосредственный (через запросы) доступ к таблицам, но разрешен доступ к этим же таблицам в диалоге с прикладной программой.

Схема доступа к данным во всех реляционных СУБД выглядит примерно одинаково и базируется на трех принципах.

1. Пользователи СУБД рассматриваются как основные действующие лица, желающие получить доступ к данным. Система управления базами данных от имени конкретного пользователя выполняет операции над БД, т.е. добавляет строки в таблицы (INSERT), удаляет строки (DELETE), обновляет данные в строках таблицы (UPDATE). Она делает это в зависимости от того, обладает ли конкретный пользователь правами на выполнение конкретных операций над конкретным объектом БД.
2. Объекты доступа — это элементы базы данных, доступом к которым можно управлять (разрешать доступ или защищать от доступа). Обычно объектами доступа являются таблицы, однако ими могут быть и другие объекты БД — формы, отчеты, прикладные программы и т.д. Конкретный пользователь обладает конкретными правами доступа к конкретному объекту.
3. Привилегии (privileges) — это операции, которые разрешено выполнять пользователю над конкретными объектами. Например, пользователю может быть разрешено выполнение над таблицей операций SELECT (ВЫБРАТЬ) и INSERT (ВКЛЮЧИТЬ).

Таким образом, в СУБД авторизация доступа осуществляется с помощью привилегий. **Установление и контроль привилегий — прерогатива администратора БД.**

Привилегии устанавливаются и отменяются специальными операторами языка SQL — GRANT (РАЗРЕШИТЬ) и REVOKE (ОТМЕНИТЬ). Оператор GRANT указывает конкретного пользователя, который получает конкретные привилегии доступа к указанной таблице.

Например, оператор GRANT SELECT, INSERT ON Bills TO bit123 устанавливает привилегии пользователю с логином bit123 на выполнение операций выбора и включения над таблицей Bills. Как видно из примера, оператор GRANT устанавливает соответствие между операциями, пользователем и объектом БД (таблицей в данном случае).

Привилегии легко установить, но легко и отменить. Отмена привилегий выполняется оператором REVOKE. Пусть, например, пользователь, имеющий логин bit123, утратил доверие администратора БД, и последний решил лишить его привилегий на включение строк в таблицу Bills. Он сделает это, выполнив оператор REVOKE INSERT ON Bills FROM bit123.

Некоторые СУБД (Oracle, Sybase) используют собственную систему паролей, в других (Ingres, Informix) идентификатор пользователя и его пароль берутся из операционной системы. Для MS SQL Server существует возможность реализации обеих политик идентификации пользователей: Windows authentication и SQL Server authentication.

Для современных БД с большим количеством пользователей актуальна проблема их объединения в группы. Традиционно применяют два способа определения групп пользователей:

- по первому способу один и тот же идентификатор используется для доступа к БД целой группы физических лиц (например, сотрудников одного отдела). Это упрощает задачу администратора БД, так как достаточно один раз установить привилегии для этого «обобщенного» пользователя. Однако такой способ в основном предполагает разрешение на просмотр, может быть на включение, но ни в коем случае — на удаление и обновление. Как только идентификатор (и пароль) становится известен большому числу людей, возникает опасность несанкционированного доступа к данным посторонних лиц;
- второй способ заключается в том, что конкретному физическому лицу присваивается уникальный идентификатор. В этом случае администратор БД должен позаботиться о том, чтобы каждый пользователь получил собственные привилегии. Если количество пользователей БД возрастает, то администратору становится все труднее контролировать привилегии. В организации, насчитывающей более 100 пользователей, решение этой задачи потребует от него много внимания.

Современные СУБД позволяют исправить эти неудобства, предлагая третий способ администрирования (Ingres, Informix, MS SQL Server). Суть его состоит в поддержке помимо идентификатора пользователя еще и идентификатора группы пользователей. Каждый пользователь кроме собственного идентификатора имеет также идентификатор группы, к которой он принадлежит. Чаще всего группа пользователей соответствует структурному подразделению организации, допустим, отделу. Привилегии устанавливаются не только для отдельных пользователей, но и для их группы.

Необходимость запуска прикладных программ пользователями, которые обладают различными правами доступа к данным, также приводит к нарушению схемы безопасности.

Одно из решений проблемы заключается в том, чтобы прикладной программе были приданы некоторые привилегии доступа к объектам БД. В этом случае пользователь, не обладающий специальными привилегиями доступа к некоторым объектам БД, может запустить прикладную программу, которая имеет такие привилегии.

В ряде СУБД это решение обеспечивается механизмом ролей (role). Роль представляет собой именованный объект, хранящийся в БД. Роль связывается с конкретной прикладной программой для придания последней привилегий доступа к БД, таблицам, представлениям и процедурам БД. Роль создается и удаляется администратором БД; для роли может быть определен пароль. Как только роль создана, ей можно предоставить привилегии доступа к объектам БД.

Пусть, например, в некоторой организации работает служащий, имеющий имя пользователя bit123. По характеру своей работы он часто обращается к таблице «ЗаработнаяПлата», он также является членом группы «Учет». Для пользователей этой группы разрешено выполнение операции SELECT над таблицей «ЗаработнаяПлата». Каждый раз, когда ему необходимо выполнить операцию выборки из таблицы, он должен для начала сеанса ввести идентификатор своей группы.

Однако ни один из пользователей группы «Учет» не может непосредственно выполнить операцию UPDATE. Для этого он должен запустить программу «КонтрольЗаработнойПлаты», которая имеет привилегии обновления этой таблицы и выполняет специальные проверки для корректного выполнения обновления. С этой целью администратором БД создается и помещается в БД роль «ОбновитьЗаработнуюПлату». Для этого он использует следующий оператор:

```
CREATE ROLE ОбновитьЗаработнуюПлату WITH PASSWORD = 'DT3110';
```

Оператор

```
GRANT SELECT, UPDATE ON ЗаработнаяПлата TO ROLE ОбновитьЗаработнуюПлату;
```

Предоставляет новой роли привилегии на выполнение операций выборки и обновления таблицы «ЗаработнаяПлата». Когда пользователь запускает программу «КонтрольЗаработнойПлаты», она получает привилегии роли «ОбновитьЗаработнуюПлату». Таким образом, данный пользователь, сам не обладая привилегиями на обновление таблицы, может выполнить эту операцию, но только запустив прикладную программу. Она же, в свою очередь, должна играть определенную роль, которой приданы соответствующие привилегии доступа к таблице.

Многопользовательский режим работы с базой данных

Система разграничения доступа и защита данных на сервере MS SQL Server реализована на нескольких уровнях:

- проверка права пользователя устанавливать соединение с сервером;
- проверка права пользователя осуществлять доступ к конкретной БД;
- проверка права пользователя манипулировать объектами БД;
- шифрование трафика между клиентом и сервером.

SQL Server поддерживает две модели проверки права пользователя устанавливать соединения с сервером:

1. **Windows only** — интегрированная модель, при которой регистрация пользователя производится на основе его текущего доменного идентификатора или идентификатора пользователя компьютера в случае, если вход был осуществлен не в домен, а на локальный компьютер. Соответственно, при таком доступе могут использоваться только trusted-соединения;
2. **SQL Server and Windows** — смешанная модель, когда допускается использование входа как на основе идентификатора пользователя Windows, так и на основе логина и пароля, передаваемого непосредственно SQL-серверу.

Выбор конкретной модели проверки прав пользователя производится на этапе установки и затем может быть изменен, например, через SQL Enterprise Manager.

После того как соединение с сервером установлено, каждому пользователю, независимо от принятой модели, ставится в соответствие учетная запись (Login). Не следует путать Login и имя учетной записи, которое вводится вместе с паролем при соединении с сервером. То, что вводится вместе с паролем, — символьное имя учетной записи (Login name). Более универсальным идентификатором учетной записи является SID (Security Identifier). SID — это бинарное значение размером до 85 байт. В свою очередь, Login name — это символьное имя, которое ему сопоставлено.

На одном сервере не может быть двух одинаковых имен учетной записи. Однако если на разных серверах существуют учетные записи с одинаковыми именами, то это не значит, что им будут сопоставлены одинаковые идентификаторы SID. В связи с этим часто возникают проблемы при переносе БД с одного сервера на другой. Лишь стандартные учетные записи имеют фиксированные (т. е. всегда одинаковые) идентификаторы SID. Для всех остальных учетных записей SID генерируются при создании.

Хорошей практикой при задании прав подключения является использование Windows Authentication и объединение пользователей в группы. Это позволяет экономить время при перемещении пользователя из группы в группу и тонко настраивать набор привилегий каждого конкретного члена группы. Кроме того, данный принцип позволяет использовать весь набор преимуществ Windows-аутентификации, таких как требование к сложности пароля, его изменение по прошествии определенного периода, время, в которое пользователь может входить в систему, и т. д.

Для сервера существует несколько серверных ролей (таблица 1). Набор серверных ролей фиксирован: нельзя ни добавить новые роли, ни удалить существующие. Каждая из этих ролей имеет свой набор прав для работы с сервером. В роли могут быть включены любые учетные записи. Члены любой роли также могут добавлять в свою роль другие учетные записи.

Таблица 1 - Список серверных ролей для MS SQL Server

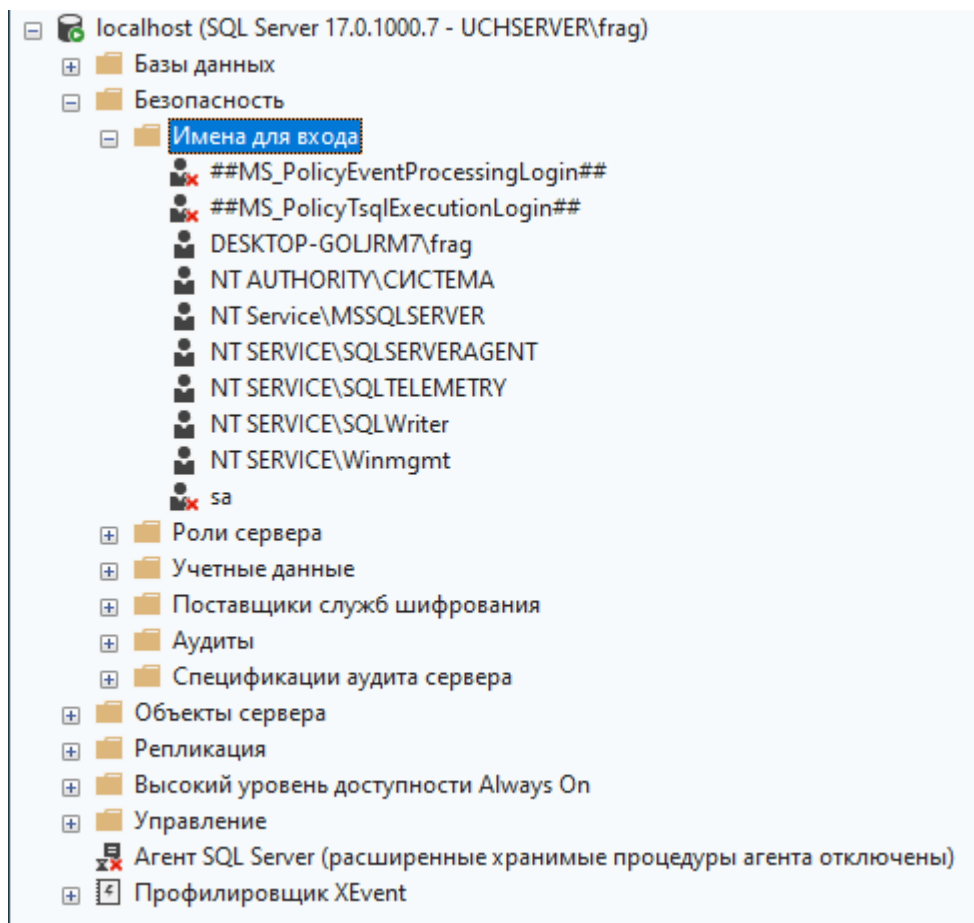
Краткое имя роли	Полное имя роли	Права
Sysadmin	System Administrators	Полные, ничем не ограниченные права на сервер
Setupadmin	Setup Administrators	Конфигурирование хранимых процедур, запускаемых при запуске сервера;

		управление связанными серверами
Serveradmin	Server Administrators	Изменение настройки сервера, остановка сервера, управление полнотекстовым поиском, управление низкоуровневыми настройками таблиц
Securityadmin	Security Administrators	Управление учетными записями (добавление, удаление, изменение пароля и БД по умолчанию и т. д.), чтение журнала ошибок сервера
Diskadmin	Disk Administrators	Используется только для совместимости с предыдущими версиями сервера
Dbcreator	Database Creators	Создание, удаление, изменение, восстановление БД
Bulkadmin	Bulk Insert Administrators	Выполнение операций массовой вставки в таблицы
Processadmin	Process Administrators	Уничтожение процессов, т. е. соединений других пользователей с сервером

В MS SQL Server существует специальная учетная запись, которая создается при установке: **sa**. Эта учетная запись использует SQL Server-аутентификацию и включена после инсталляции сервера в серверную роль sysadmin.

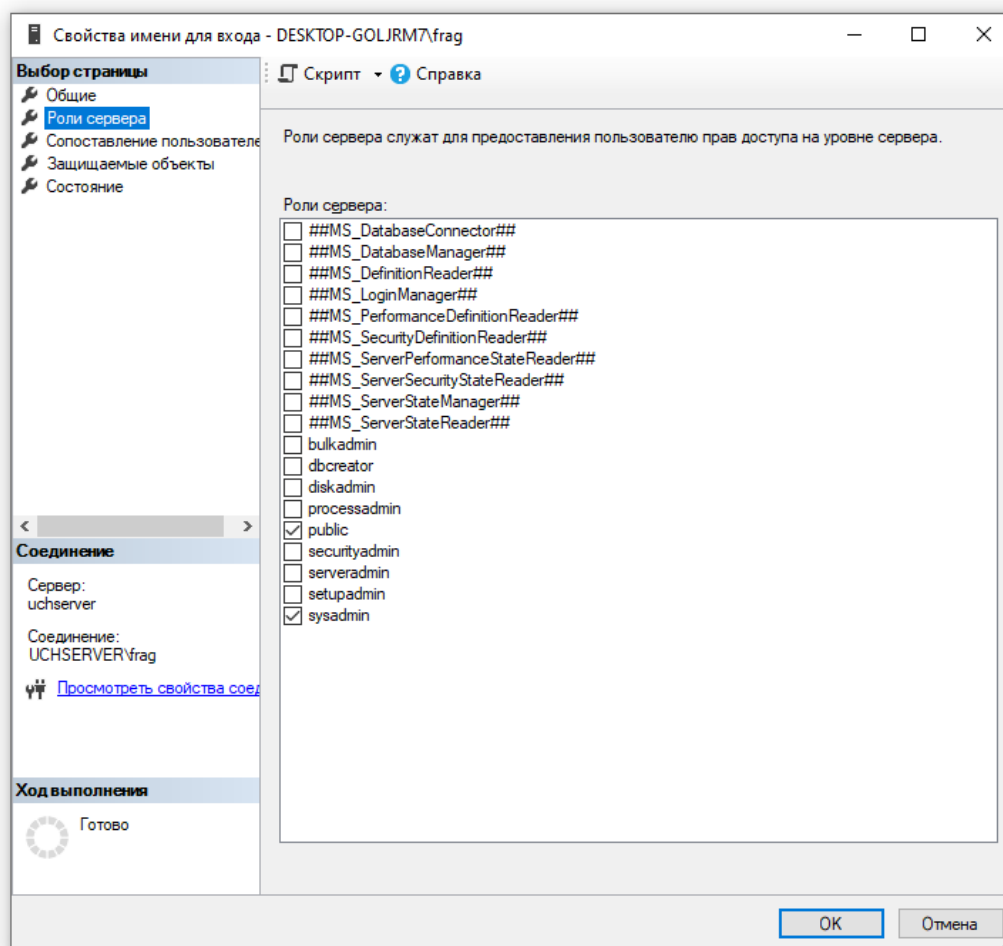
Пользователя sa рекомендуется удалять из соображений безопасности (предоставив предварительно администраторские права какому-либо другому пользователю). Как минимум, требуется, чтобы его пароль был не пустым, иначе безопасность сервера окажется под угрозой.

Перечень всех серверных ролей можно посмотреть с помощью графического интерфейса **Enterprise Manager**. Для этого нужно открыть папку **Безопасность (Security)**, установить курсор на папке **Роли сервера (Server Roles)**, и тогда в правом окне появится список серверных ролей с указанием их прав (рисунок ниже).



Стандартные серверные роли

Чтобы увидеть в какую серверную роль входит пользователь с определенным именем входа в SQL Server, нужно выбрать логин в списке логинов папки **Имена для входа (Безопасность)**, щелчком правой клавиши мыши вызвать контекстное меню, выбрать в нем пункт **Свойства (Properties)** и перейти на закладку **Роли сервера (Server roles)** см. рисунок ниже.



Серверные роли конкретного пользователя

Права на доступ к базе данных

Факт установления соединения с сервером еще не дает пользователю права осуществлять манипуляции с объектами сервера.

Для каждой БД SQL Server хранит независимый набор пользователей и групп, в которые они входят. По умолчанию в каждой базе существует пользователь dbo — владелец БД, и группа public, к которой принадлежат все имена пользователей этой базы. Каждое имя пользователя может дополнительно принадлежать еще и к другим группам. Выделяются фиксированные роли БД, обладающие стандартным набором прав. Могут быть созданы дополнительные группы и им назначены права. Набор фиксированных ролей БД приведен в таблице 1.

Таблица 1 - Стандартные роли пользователей баз данных

Имя роли	Права
Public	Используется для задания прав по умолчанию; в нее входят все пользователи БД
Db_securityadmin	Члены роли могут управлять правами доступа других пользователей к объектам БД и членством их в ролях
Db_owner	Члены роли имеют права владельца, т. е. могут выполнять любые действия с БД
Db_denydatawriter	Членам роли запрещено изменение данных, даже в том случае, если

	был явно предоставлен доступ к каким-либо данным
Db_denydatareader	Членам роли запрещен просмотр данных, даже в том случае, если был явно предоставлен доступ к каким-либо данным
Db_datawriter	Пользователи, включенные в эту роль, могут изменять данные в любой таблице или представлении БД
DB_datareader	Пользователи, включенные в эту роль, могут читать данные из любой таблицы и представления БД
Db_ddladmin	Члены роли могут создавать, изменять и удалять объекты БД
Db_backupoperator	Члены роли выполняют резервное копирование БД
Db_accessadmin	Члены роли имеют права создать, изменять, удалять пользователей БД

Каждой роли и (или) каждому пользователю БД могут быть назначены права на любой из объектов БД (чтение, запись, изменение, удаление данных, выполнение ссылочной целостности и выполнение процедур). Если имеется несколько пользователей с одинаковыми правами, то их следует объединять в группы и назначать права этой группе.

Чтобы конкретный пользователь имел доступ к БД, его учетная запись должна быть ассоциирована с каким-либо пользователем в этой БД. Одна учетная запись может соответствовать в разных БД разным пользователям. В одной БД одна учетная запись может соответствовать только одному пользователю. Один пользователь БД может соответствовать не более чем одной учетной записи.

Если учетная запись не сопоставлена никакому пользователю БД, доступ к объектам БД можно получить, если в ней создан специальный пользователь — **Guest**. Под этим именем доступ к объектам БД получают те, чья учетная запись не сопоставлена конкретному пользователю.

Учетные записи, входящие в серверную роль sysadmin, всегда сопоставляются с пользователем dbo даже в том случае, если базу создавал какой-либо другой пользователь сервера. Кроме того, пользователям, применяющим Windows Authentication, могут быть предоставлены права на объекты БД напрямую, без предварительного сопоставления с пользователем БД. Однако делать этого не рекомендуется.

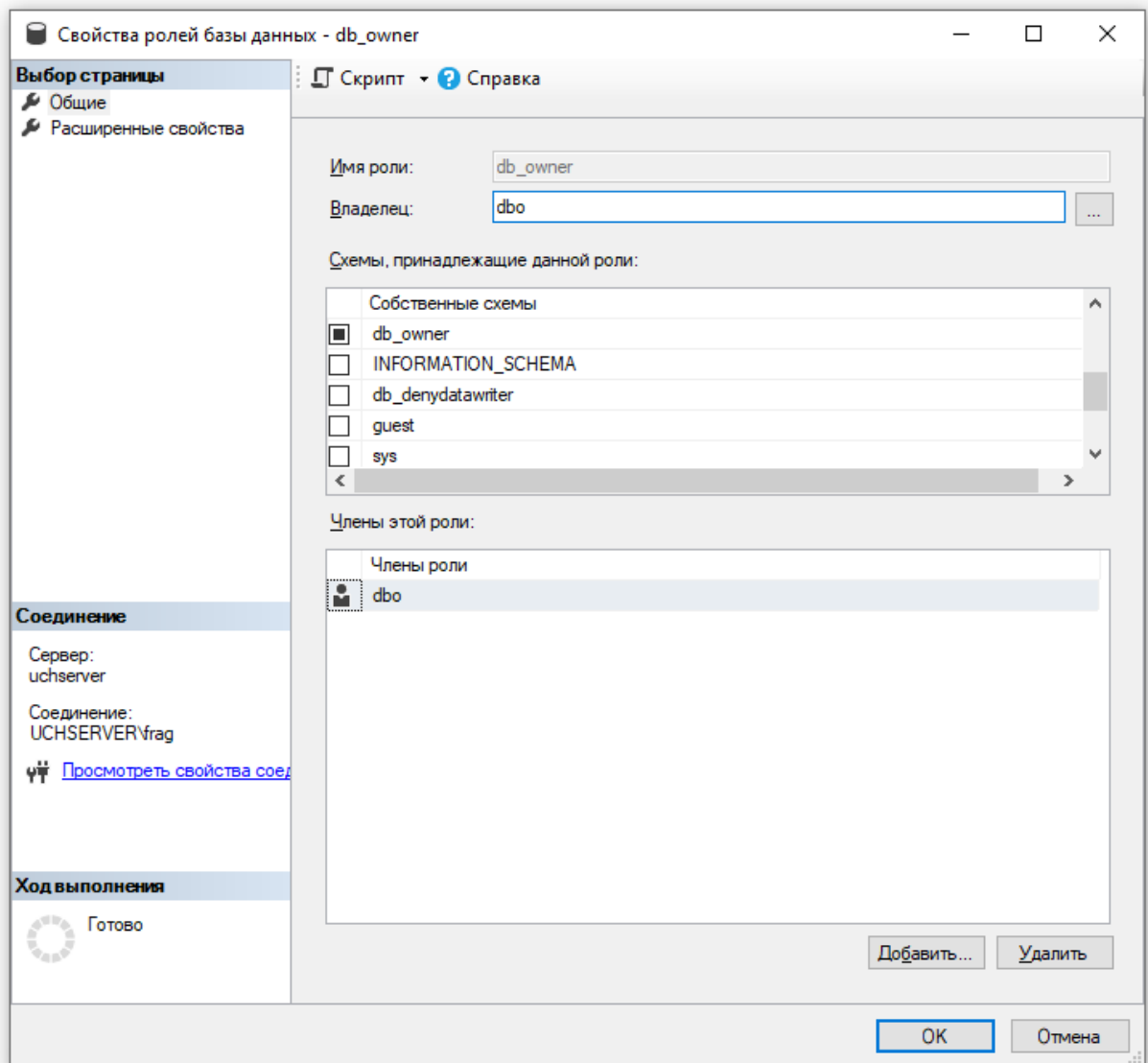
В таблице 2 приведены команды, представления и функции, предназначенные для работы с ролями уровня БД.

Таблица 2 - Средства для работы с ролями базы данных

Компонент	Тип	Description
sp_helpdbfixedrole	Метаданные	Возвращает список всех предопределенных ролей БД
sp_dbfixedrolepermission	Метаданные	Отображает разрешения предопределенной БД
sp_helprole	Метаданные	Возвращает информацию о ролях, относящихся к текущей БД
sp_helprolemember	Метаданные	Возвращает сведения о членах роли в текущей БД
sys.database_role_members	Метаданные	Возвращает одну строку для каждого члена каждой роли БД
IS_MEMBER	Метаданные	Указывает, является ли текущий пользователь членом указанной группы Microsoft Windows или

		роли БД Microsoft SQL Server
CREATE ROLE	Command	Создает новую роль БД в текущей БД
ALTER ROLE	Command	Изменяет имя или членство роли БД
DROP ROLE	Command	Удаляет роль из БД
sp_addrole	Command	Создает новую роль БД в текущей БД
sp_droprole	Command	Удаляет роль БД из текущей БД
sp_addrolemember	Command	Добавляет пользователя БД, роль БД, имя входа Windows или группу Windows к роли текущей БД. Все платформы, за исключением параллельного хранилища данных, должны использовать вместо этого ALTER ROLE
sp_droprolemember	Command	Удаляет учетную запись безопасности из роли SQL Server в текущей БД. Все платформы, за исключением параллельного хранилища данных, должны использовать вместо этого ALTER ROLE
GRANT	Permissions	Добавляет разрешение для роли
DENY	Permissions	Запрещает разрешение для роли
REVOKE	Permissions	Удаляет разрешения, выданные или запрещенные ранее

Посмотреть текущее состояние ролей БД можно, выбрав в объектах БД **Роли** и щелкнув на любой роли в правом окне. В результате отобразится список пользователей, которые принадлежат к данной роли в конкретной БД (рисунок ниже).



Список пользователей, имеющих конкретную роль в БД

Одновременно в данном интерфейсе можно **добавить к роли нового пользователя**. Однако необходимо помнить, что этот **пользователь уже должен иметь право на подключение к серверу БД**, т. е. быть легальным пользователем сервера MS SQL Server, на котором размещена БД, и быть пользователем самой БД.

Права на доступ к объектам базы данных

Следующий уровень безопасности — это разрешения на уровне объектов БД. Существует несколько видов разрешений на объекты БД.

Во-первых, это разрешения на операции SELECT, UPDATE, INSERT и DELETE (операции выборки, изменения, вставки и удаления). Эти разрешения применимы только для таблиц и представлений.

Для хранимых процедур и определенных пользователями функций существует разрешение на выполнение (EXECUTE). Если в отношении некоторой процедуры или функции для пользователя предоставлено такое разрешение, то он может ее выполнять, иначе — нет.

Наконец, существует **разрешение на декларативную целостность** (Declarative Referential Integrity — **DRI**). Такие разрешения допускаются для таблиц, представлений и

пользовательских функций. Если пользователь имеет разрешение на DRI для некоторой таблицы, это означает, что он может создавать в других таблицах связи, ссылающиеся на ту таблицу, для которой пользователь имеет это разрешение.

Кроме того, при **создании представления можно указать, что это представление привязано к схеме данных**. Это позволяет отслеживать попытки изменения объектов, на основе которых строится представление или функция. Например, если представление строится на основе таблицы Students и используется привязка этого представления к схеме, то будет невозможно удалить таблицу Students, пока такая привязка существует. Также будет невозможно изменить ту часть таблицы (столбцы), которая присутствует в описании представления. Для того чтобы осуществить привязку представления или функции к схеме, необходимо, чтобы на все ссылающиеся объекты для пользователя, создающего или изменяющего данное представление или функцию, имелись разрешения типа DRI.

Разрешения типа DRI для представлений и пользовательских функций имеют такой же смысл, как и для таблиц, за исключением того, что они не дают возможности устанавливать связи, поскольку ни с представлениями, ни с функциями нельзя установить связь посредством внешнего ключа. Таким образом, для представлений и пользовательских функций разрешения типа DRI позволяют привязывать к схеме объекты, которые основываются на данных представлениях или функциях.

В MS SQL Server существует три состояния возможности доступа: **разрешение (GRANT), неявное отклонение доступа (REVOKE) и запрет (DENY)**.

Оператор GRANT имеет следующий синтаксис:

```
GRANT {<список действий> | ALL PRIVILEGES}  
ON <имя_объекта> TO {<имя_пользователя> | PUBLIC}  
[WITH GRANT OPTION]
```

Список действий **определяет набор действий** из общедопустимого перечня действий над объектом данного типа:

- опция ALL PRIVILEGES указывает, что разрешены все действия из допустимых для объектов данного типа;
- <Имя_объекта> задает имя конкретного объекта: таблицы, представления, хранимой процедуры, триггера;
- <Имя_пользователя> или PUBLIC определяет, кому предоставляются данные привилегии;
- опция WITH GRANT OPTION является необязательной. Она определяет режим, при котором передаются не только права на указанные действия, но и право передавать (делегировать) эти права другим пользователям. Передавать права в этом случае пользователь может только в рамках разрешенных ему действий.

Для отмены ранее назначенных привилегий в стандарте SQL определен оператор REVOKE. Оператор отмены привилегий имеет следующий синтаксис:

```
REVOKE {<список операций> | ALL PRIVILEGES}  
ON <имя_объекта>  
FROM {<список пользователей> | PUBLIC} {CASCADE | RESTRICT}
```

Опции CASCADE или RESTRICT определяют, каким образом должна производиться отмена привилегий. Опция CASCADE отменяет привилегии не только того пользователя,

который непосредственно упоминался в операторе GRANT при предоставлении ему привилегий, но и всем пользователям, которым были присвоены привилегии данным пользователем с использованием данной ему опции WITH GRANT OPTION.

С разрешением и запретом все интуитивно понятно: происходит либо предоставление некоторых прав пользователю, либо выставление запретов. Что касается неявного отклонения доступа, можно считать, что это удаление ранее предоставленного разрешения или запрета для данного пользователя. Если бы не было возможности неявного отклонения доступа, то для некоторого конкретного пользователя можно было бы либо дать разрешение, либо выставить запрет на некоторый конкретный объект БД. С помощью же неявного отклонения доступа можно убрать как явное разрешение, так и явное отклонение доступа, т. е. установить неопределенность. Такое состояние называется неявным отклонением доступа. Дело в том, что здесь применяется политика, в соответствии с которой запрещается все, что явно неразрешено, т. е. если у пользователя не будет ни запрета, ни разрешения на некоторую операцию, то он не сможет ее выполнить.

Для одного и того же объекта на одну и ту же операцию пользователь может иметь сразу несколько разрешений (или запретов), т. е. непосредственно пользователю, конечно, может быть выдано только одно разрешение. Однако пользователь может входить в множество различных ролей БД. Например, для БД института можно рассмотреть роли «Преподаватели», «Студенты», «Сотрудники деканата», «Сотрудники профсоюза», «Лаборанты», «Системные администраторы». Один пользователь может входить сразу в несколько ролей. Для некоторого объекта на одну операцию (например, операцию удаления) могут быть выставлены и запреты, и разрешения. Кроме того, они могут быть выставлены еще дополнительно непосредственно для каждого пользователя.

Таким образом, каждый пользователь БД может иметь разрешения, предоставленные непосредственно ему или посредством вхождения в роли. Причем может получиться, что эти разрешения и запреты будут противоречить друг другу. Например, операция удаления будет запрещена для роли «Студенты», но разрешена для роли «Лаборанты». Если учесть, что некоторый пользователь может входить в обе роли, как будет определяться возможность или невозможность выполнить ту или иную операцию в таких случаях? Наиболее весомым считается запрет из тех соображений, что главное — не допустить (по ошибке) несанкционированного доступа к данным. Если же кто-либо из пользователей недополучит каких-либо прав, то он обратится к администратору и тот предоставит ему необходимые права, т. е. считается, что лучше недодать прав, чем дать их слишком много. В этом и состоит разница между REVOKE и DENY: если пользователю одновременно даны разрешение (GRANT) и запрет (DENY), то результатом будет запрет, а если же дополнительно к разрешению дано неявное отклонение доступа, то результатом будет разрешение.

Интерпретация двойных назначений разрешений приведена в таблице 3. Из анализа данных, приведенных в таблице, можно сделать вывод, что будет происходить при наличии разных разрешений.

Таблица 3 - Интерпретация двойных назначений разрешений

Разрешение 1	Разрешение 2	Результат	Итоговое разрешение
GRANT	DENY	DENY	Нет
GRANT	REVOKE	GRANT	Есть
GRANT	GRANT	GRANT	Есть
DENY	DENY	DENY	Нет
DENY	REVOKE	DENY	Нет
DENY	GRANT	DENY	Нет

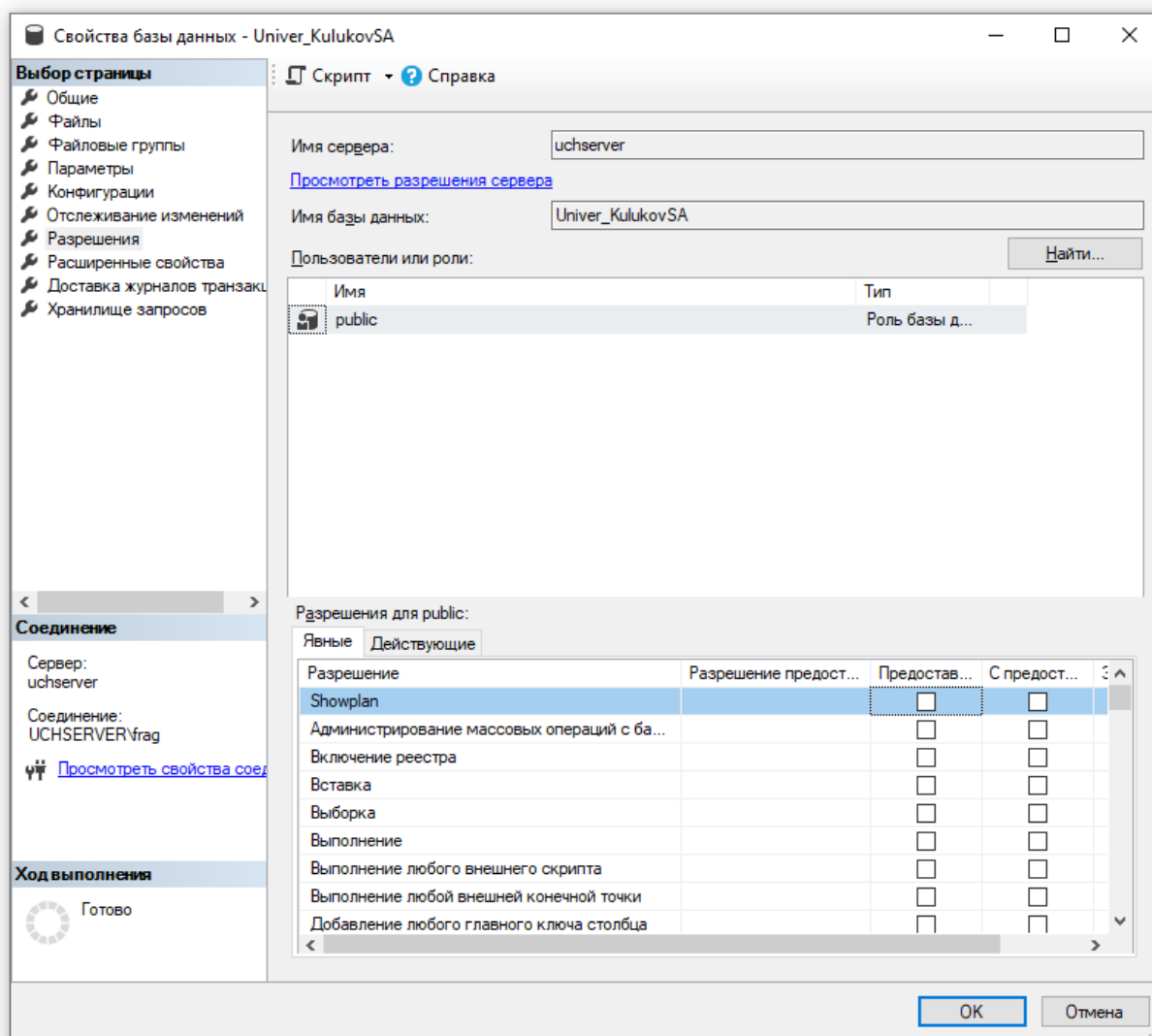
REVOKE	DENY	DENY	Нет
REVOKE	REVOKE	REVOKE	Нет
REVOKE	GRANT	GRANT	Есть

Выдать разрешения на объект БД может либо владелец объекта, либо владелец БД, либо любой другой пользователь, которому уже предоставлены эти права с дополнительной опцией, позволяющей выдавать это разрешение кому-либо еще.

Владельцем объекта БД является пользователь, который его создал. В принципе, можно передавать владение объектом БД другому пользователю, но это используется нечасто.

Назначение прав пользователям БД, так же как и большинство функций администрирования, может быть выполнено с использованием **графического интерфейса Enterprise Manager**. Для этого необходимо установить курсор на конкретного пользователя или на пользовательскую роль в БД, если нужно предоставить одинаковые права всем членам данной роли, и нажать кнопку **Разрешения**.

В открывшемся окне нужно установить «галочки» в соответствующие графы разрешений. Каждая графа отвечает за разрешение определенных действий над определенным объектом.



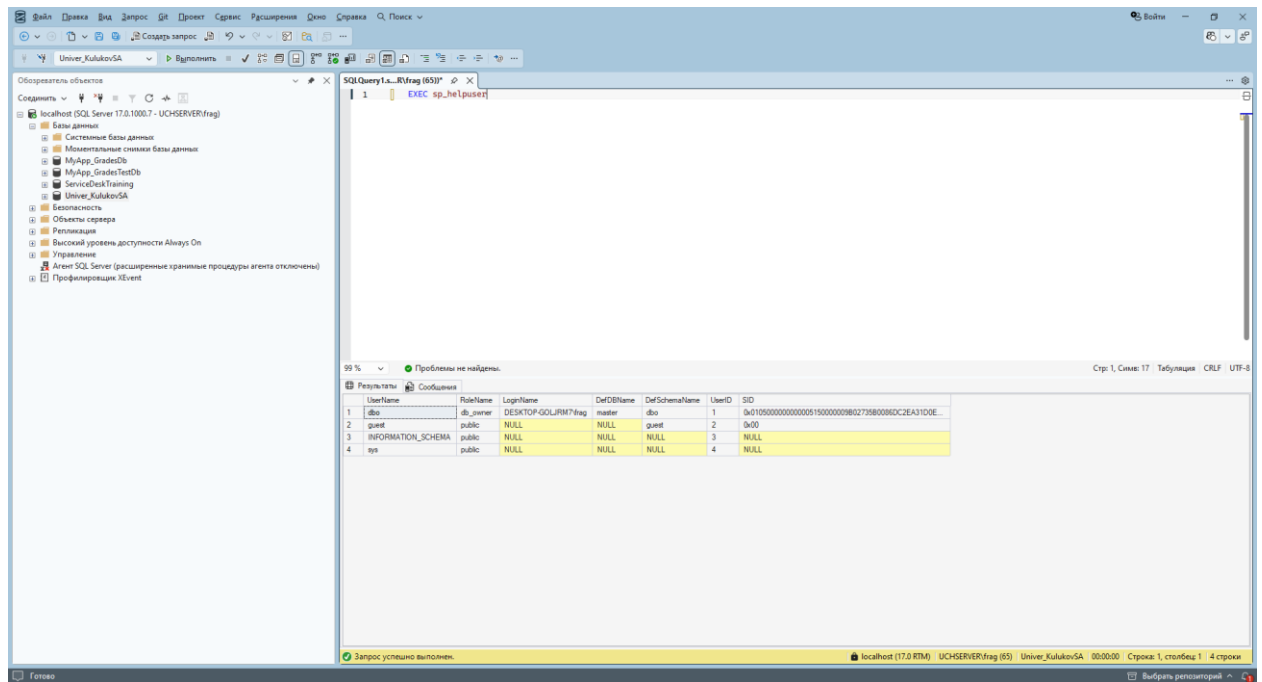
Интерфейс предоставления прав пользователям

Перечень доступных операций зависит от типа объекта. Если требуется еще более тонко дифференцировать права доступа, то можно воспользоваться кнопкой **Столбца** и установить права на работу с отдельными атрибутами.

Управление пользователями баз данных

Удобным механизмом получения информации о пользователях текущей БД является системная хранимая процедура **sp_helpuser**:

```
USE AdventureWorks2014;
EXEC sp_helpuser
```



Результат выполнения системной процедуры `sp_helpuser`

Для первого пользователя указана только часть идентификатора безопасности SID, так как размер страницы не позволил вместить его весь.

CREATE USER добавляет нового пользователя в текущую БД. Далее приведены несколько типов пользователей.

Типы пользователей:

1. **пользователи на основе имен входа в БД master** — наиболее распространенный тип пользователя. Пользователь с именем входа для проверки подлинности SQL Server:

```
CREATE USER Mary;
```

2. **пользователи, которые проходят проверку подлинности SQL Server.** Проверка рекомендуется для того, чтобы повысить переносимость БД. Допускается только в автономной БД в SQL Server.

```
CREATE USER Mary WITH PASSWORD = '*****';
```

DROP USER — удаляет пользователя из БД.

```
DROP USER user_name;
```

Здесь `user_name` указывает имя, по которому пользователь идентифицируется в этой БД.

Пользователи, которые владеют защищаемыми объектами, не могут быть удалены из БД. Перед удалением пользователя, который владеет защищаемым объектом, необходимо удалить или сменить владельца защищаемого объекта.

Пользователь **Guest** не может быть удален, однако его можно отключить, отменив разрешение CONNECT с помощью инструкции REVOKE CONNECT FROM GUEST в любой БД, кроме **master** или **tempdb**.

Инструкция CREATE ROLE создает новую роль БД в текущей БД:

```
CREATE ROLE имя_роли [AUTHORIZATION имя_владельца]
```

где **имя_роли** — имя создаваемой роли; AUTHORIZATION `имя_владельца` — пользователь (или роль) БД, который станет владельцем новой роли.

Если пользователь не указан, владельцем роли станет пользователь, выполнивший инструкцию CREATE ROLE.

Роли — сущности, защищаемые на уровне БД.

После создания роли необходимо настроить для нее разрешения уровня БД с помощью инструкций GRANT, DENY и REVOKE. Чтобы добавить членов роли БД, используется команда ALTER ROLE (Transact-SQL).

Роли БД видны в представлениях каталога **sys.database_role_members** и **sys.database_principals**.

Для передачи роли во владение другому пользователю необходимо связанное с этим пользователем разрешение IMPERSONATE.

Для передачи роли во владение другой роли необходимо членство в роли-получателе или связанное с этой ролью разрешение ALTER.

Для передачи роли во владение роли приложения необходимо связанное с прикладной ролью разрешение ALTER.

В примере, приведенном далее, показано **создание роли БД buyers, принадлежащей пользователю BenMiller**.

```
USE AdventureWorks2024;  
CREATE ROLE buyers AUTHORIZATION BenMiller;  
GO
```

Следующий пример создает роль БД **auditors, принадлежащую предопределенной роли БД db_securityadmin**.

```
USE AdventureWorks2024;  
CREATE ROLE auditors AUTHORIZATION db_securityadmin;  
GO
```

Следующий пример создает роль БД **buyers, принадлежащую пользователю BenMiller**.

```
USE AdventureWorks2024;  
  
CREATE ROLE buyers AUTHORIZATION BenMiller;  
GO
```

Еще один пример создает роль БД **auditors**, принадлежащую предопределенной роли БД **db_securityadmin**.

```
USE AdventureWorks2024;  
  
CREATE ROLE auditors AUTHORIZATION db_securityadmin;  
GO
```

Первоначально роль является пустой, т. е. не имеющей ни одной привилегии. Включение привилегий в роль производится командой **GRANT**. Роли могут создавать администратор и те пользователи, которым такое право предоставил администратор. Тот, кто создал роль, или тот, кто явно указан при определении роли (стандарт **SQL**), обладает правами ее администрирования, т. е. может:

- предоставлять роль другому пользователю или роли;
- аннулировать роль для другого пользователя или роли;
- изменять определение роли, например, в целях изменения авторизации доступа к ней;
- удалять роль.

Как и любой другой объект БД, определение роли можно изменять с помощью команды **ALTER ROLE** и удалять с помощью команды **DROP ROLE**:

```
DROP ROLE имя_роли
```

Предоставление привилегий производится двумя предложениями. Одно предоставляет привилегии на объекты БД, а другое — на роли.

Предоставление привилегий на объекты производится следующим предложением:

```
GRANT {ALL PRIVILEGES | список_привилегий} ON объект  
TO {PUBLIC | {пользователь | роль}...}  
[WITH HIERARCHY OPTION]  
[WITH GRANT OPTION]  
[GRANTED BY {CURRENT_USER | CURRENT_ROLE}]
```

К множеству объектов, для которых определяются привилегии, относят:

- таблицы и представления БД;
- домены, на которых определяются типы столбцов;
- определяемые типы данных;
- последовательности;
- программы, вызываемые из **SQL**.

Список привилегий, предоставляемых для работы с объектами БД, включает в себя:

- использование предложения SELECT ко всей таблице или только к отдельным ее столбцам;
- использование предложения DELETE;
- использование предложения INSERT ко всей таблице или только к отдельным ее столбцам. В этом случае одни пользователи могут вводить строки таблицы только со значениями отдельных столбцов, а другие — затем обновить значения других столбцов;
- использование предложения UPDATE ко всей таблице или только к отдельным ее столбцам;
- ссылки на таблицы или их столбцы любым способом — привилегия REFERENCES, которая позволяет ссылаться на столбцы во внешнем ключе или при задании ограничения целостности CHECK;
- использование некоторых из объектов БД способом, отличающимся от описанных ранее, — привилегия USAGE. Если пользователь получит такую привилегию по отношению, например, к домену, он может использовать его для указания типа столбца определяемой таблицы. В противном случае он этого сделать не может;
- инициирование триггеров из указанной таблицы — привилегия TRIGGER;
- выполнение программы, вызываемой из SQL, — привилегия EXECUTE.

Дополнительные фразы предложения GRANT означают следующее:

- **WITH HIERARCHY OPTION** — передает права указанного объекта всем его подобъектам. Она имеет смысл только для привилегии SELECT, предоставленной объекту в иерархической структуре объектов (иерархическая структура наследования в объектно-ориентированном SQL). В этом случае специфицируемая привилегия распространяется на все производные объекты того объекта, относительно которого специфицировано это полномочие;
- **WITH GRANT OPTION** — означает, что указанному пользователю не только предоставляются специфицированные привилегии, но и права предоставлять эти привилегии другим пользователям;
- **GRANTED BY** — указывает, кто именно передает полномочия; могут быть указаны либо текущий пользователь, либо текущая роль.

Несколько примеров:

```
GRANT SELECT, UPDATE (Salary. Rise) ON Teacher TO Kostya;
GRANT ALL PRIVILEGES ON Room TO Jones WITH GRANT OPTION;
GRANT SELECT ON Faculty TO Public;
DELETE UPDATE ON Subject TO Ivanov, Petrov;
```

GRANT REFERENCES (DepPK). UPDATE (Name. Head, Building) ON Depatment TO Blake;

Предоставление привилегий на роли задается следующим предложением:

```
GRANT список_ролей
TO {PUBLIC | {пользователь | роль} ...}
[WITH ADMIN OPTION]
[GRANTED BY {CURRENT_USER | CURRENT_ROLE}]
```

Предоставление роли имеет три варианта:

1. **PUBLIC** — все имеющиеся в указанных ролях привилегии оказываются доступными всем пользователям;
2. **пользователь** — получает все привилегии, которые имеются в указанных ролях;
3. **роль** — получает дополнительно все привилегии указанных ролей.

Фраза WITH ADMIN OPTION указывает, что вторая роль приобретает все права по администрированию первой роли (т. е. предоставлять или отменять ее другим ролям или пользователям, изменять список ее полномочий или даже вообще удалять эту роль). Фраза GRANTED BY имеет тот же смысл, что и в предыдущем предложении.

Роль — единственный системный объект, на который можно предоставлять пользователям привилегии по их использованию:

```
GRANT SpecialRole TO Smith;
```

Здесь пользователь Smith получает все полномочия, имеющиеся в роли Special Role;

```
GRANT Special Role TO AccountGroupRole;
```

Здесь вторая роль получает все те привилегии, которые имеются в первой роли;

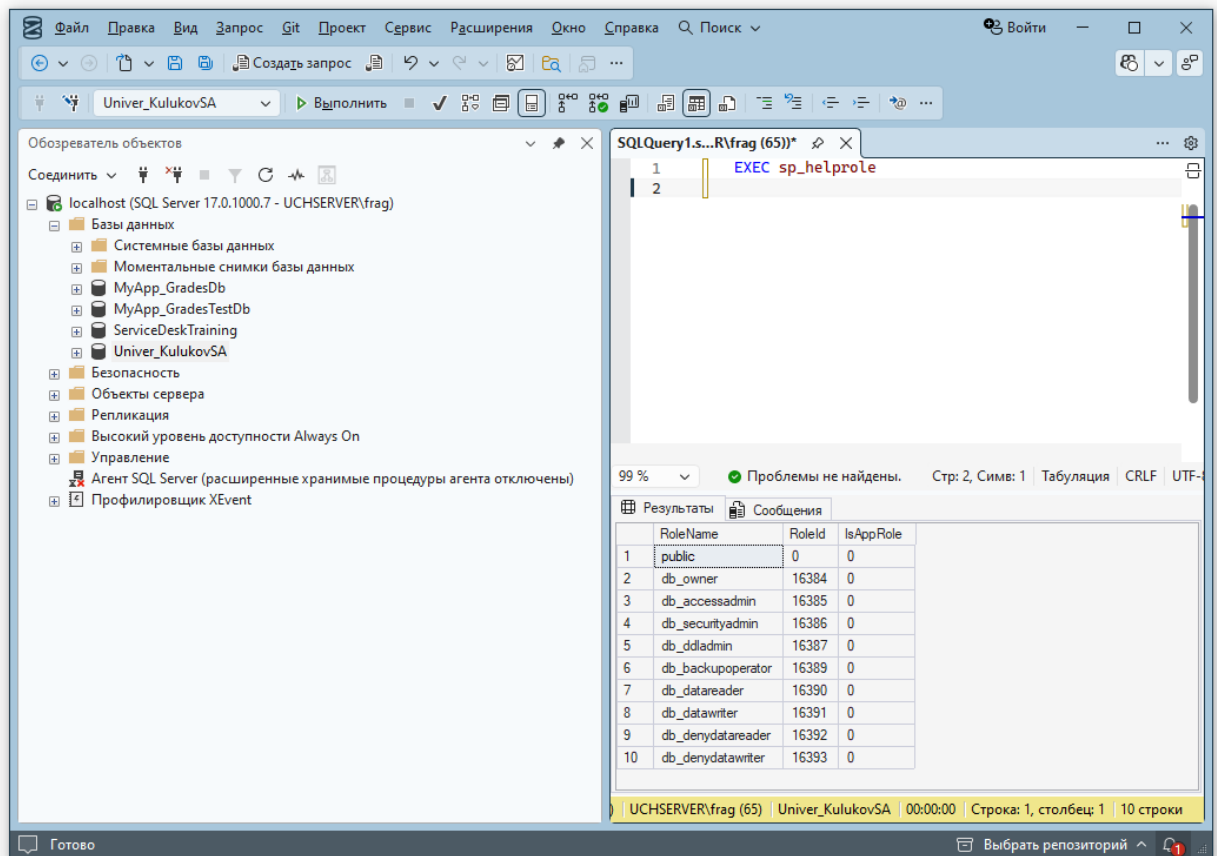
```
GRANT SpecialRole TO Public;
```

Все пользователи получают полномочия роли Special Role.

Для получения информации о ролях БД можно применять системную хранимую процедуру sp_helprole:

```
USE AdventureWorks2014;  
EXEC sp_helprole
```

Будет возвращен результат, показанный на рисунке ниже.



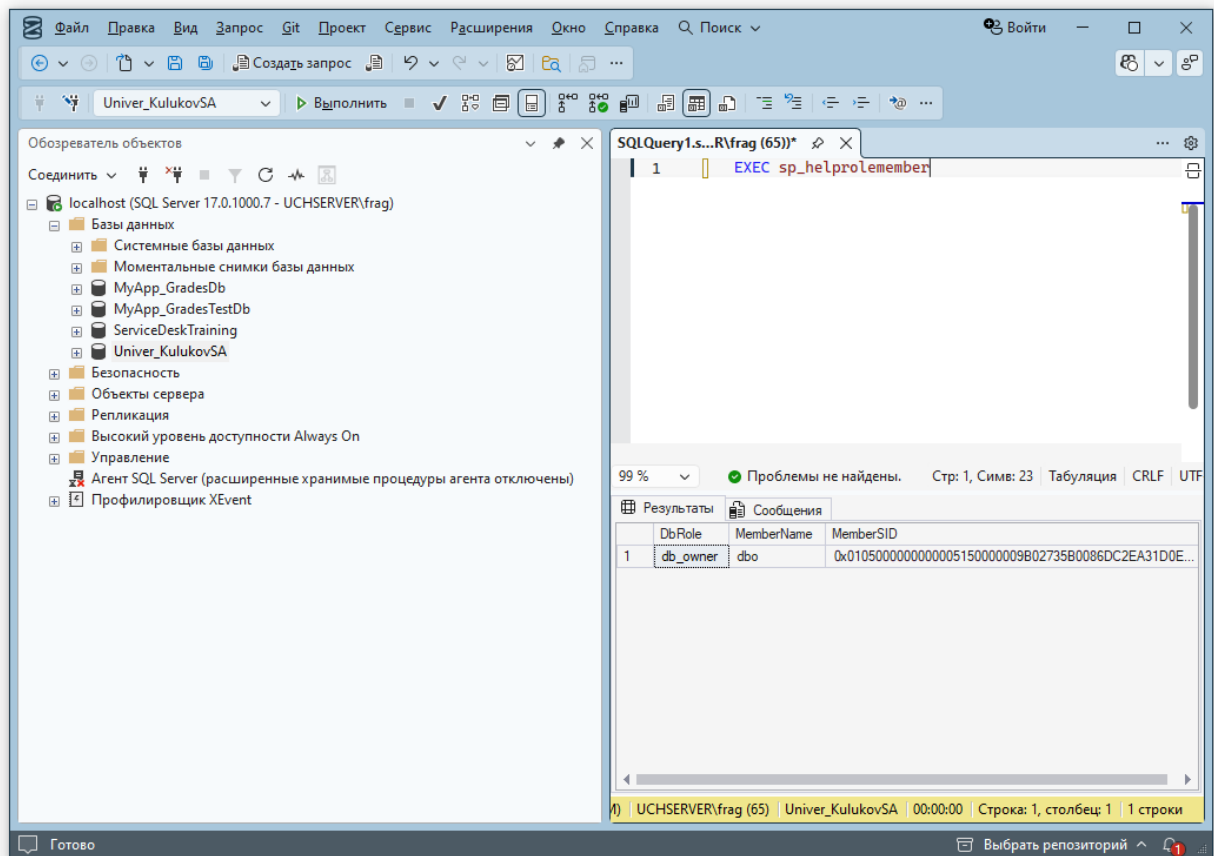
Результат выполнения процедуры sp_helprole

Как видно, процедура выводит сведения об имени роли, ее идентификаторе и флаг, говорящий о том, является ли соответствующая роль ролью приложения.

Получение информации о членстве во всех ролях текущей БД осуществляется следующим образом:

```
USE AdventureWorks2024;
EXEC sp_helprolemember
```

Будет получен результат, показанный на рисунке ниже.



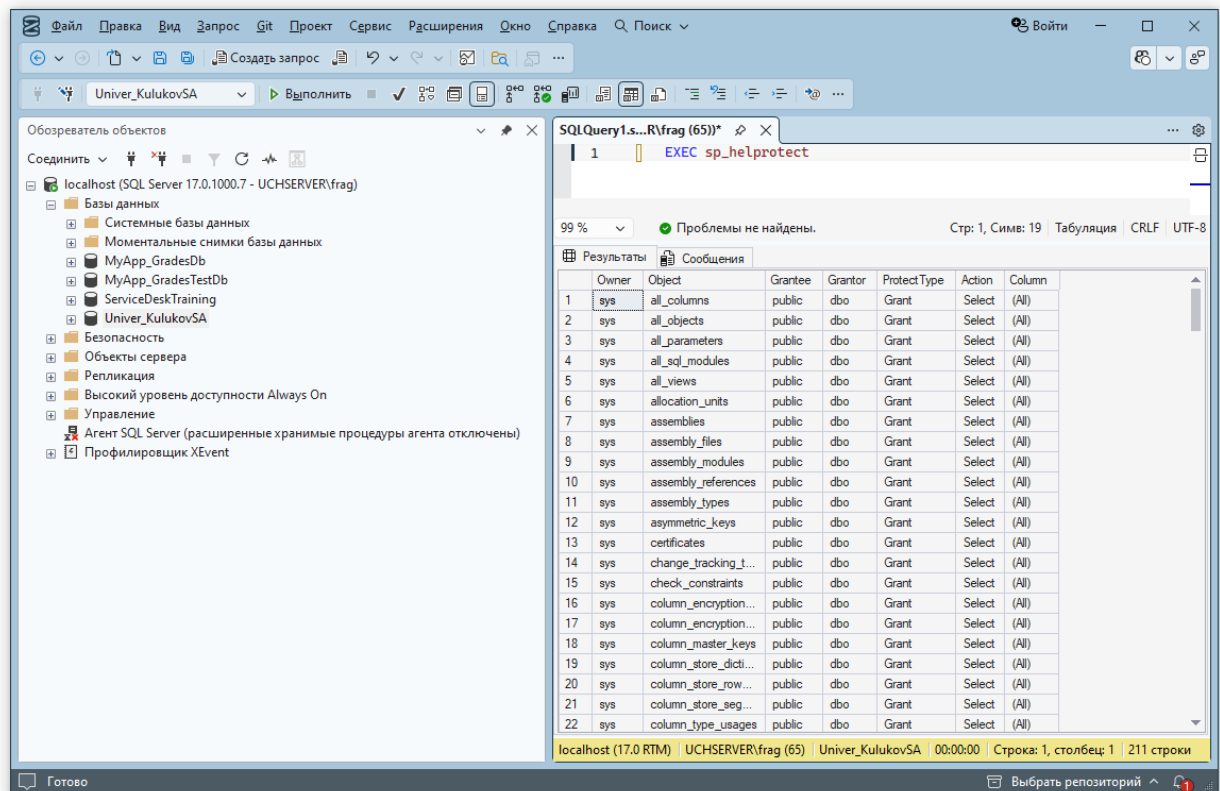
Результат выполнения процедуры sp_helprolemember

Получив набор прав, пользователь сможет выполнять разрешенные действия. Но в некоторых случаях бывает необходимо запретить ему выполнять некоторые операции в БД.

В качестве иллюстрации использования хранимой процедуры sp_helpprotect получим сведения о правах доступа пользователей к таблице authors:

```
USE AdventureWorks2024;
EXEC sp_helpprotect
```

Будет получен результат, показанный на рисунке ниже.



Результат выполнения процедуры `sp_helprotect`

Информация выводится в нескольких столбцах:

- **Owner** — в этом столбце указывается имя текущего владельца объекта;
- **Objects** — имя объекта, о котором выводится информация;
- **Grantee** — имя пользователя или роли, которым выдано разрешение на доступ к соответствующему объекту;
- **Grantor** — имя пользователя или роли БД, который предоставил другому пользователю соответствующее право доступа;
- **Protect Type** — тип разрешения, выданного пользователю;
- **Action** — тип действия, которое разрешено выполнять пользователю;
- **Column** — если разрешение выдано для определенного столбца таблицы, то будет выведено имя этого столбца. Если разрешение выдано для всех столбцов, то будет указано [All + New].

Пользователь, создавший таблицу (а для этого он должен иметь такое право), **является владельцем этой таблицы**. Это означает, что он имеет абсолютно все права на эту таблицу, на внешнюю память, вплоть до передачи прав на нее другим пользователям. **Пользователь, создавший представление, становится его владельцем**. Но не все привилегии по отношению к нему он получает автоматически. Для создания представления необходимо иметь привилегию SELECT ко всем таблицам. Таким образом, владелец представления получает такую привилегию автоматически. Все другие привилегии он может получить посредством явного предоставления таковых по отношению ко всем базовым таблицам представления.

Отмена ранее переданных привилегий производится командой REVOKE. В стандарте SQL имеется две разновидности этой команды: для отмены полномочий на объекты и отмены роли. **Первая разновидность команды REVOKE имеет следующий синтаксис:**

```
REVOKE [{GRANT | HIERARCHY} OPTION FOR]
{ALL PRIVILEGES | список_привилегий} ON объект FROM {PUBLIC | {пользователь | роль}...}
[GRANTED BY {CURRENT_USER | CURRENT_ROLE}] {CASCADE | RESTRICT}
```

Это предложение имеет структуру, аналогичную структуре предложения GRANT: оно содержит набор отменяемых привилегий, объект, относительно которого отменяются привилегии, и пользователей, у которых отменяются привилегии. С помощью этой команды можно отменить все ранее предоставленные привилегии или только некоторые. Фраза HIERARCHY OPTION FOR отменяет ту разновидность привилегии, которая предоставляется предложением GRANT с использованием фразы WITH HIERARCHY OPTION. Фраза GRANTED BY имеет тот же смысл, что и в предложении GRANT.

Любой пользователь может отменять только те привилегии, которые он ранее передавал, но не может отменить полномочия, которыми обладает сам.

Многие пользователи могут предоставить одну и ту же привилегию на объект одному и тому же пользователю (или роли). Для аннулирования у пользователя такой привилегии необходимо, чтобы ее отменили все пользователи, которые ее ранее передали. Если хотя бы один из пользователей не отзывet такую привилегию, то ее получатель будет ею обладать.

Например, пользователь Dep_Head предоставил пользователю Andrew полномочия выбирать данные из таблицы Teacher, а также выбирать и обновлять таблицу Sgroup:

```
GRANT SELECT, UPDATE ON Sgroup TO Andrew;
GRANT SELECT ON Teacher TO Andrew;
```

После этого еще один пользователь, Dep_Deputy, предоставил Andrew возможность выбирать и удалять данные из таблицы Subject и выбирать данные из таблицы Subject:

```
GRANT SELECT, DELETE ON Sgroup TO Andrew;
GRANT SELECT ON Subject TO Andrew;
```

Andrew получил привилегии на таблицу Sgroup от двух лиц, причем привилегия SELECT получена от обоих пользователей. Чуть позже пользователь Dep_Head решил отменить все ранее переданные им привилегии пользователю Andrew. В связи с этим он инициирует следующее предложение:

```
REVOKE ALL PRIVILEGES ON Sgroup FROM Andrew;
```

В связи с этим Andrew лишился права обновлять таблицу Sgroup, однако выбирать из нее данные и удалять их он все же может, так как такое право у него осталось от пользователя Dep_Deputy.

Возможность передавать привилегии вместе с правами на передачу их другим приводит к возможности порождения целой цепочки порожденных привилегий. Например, пользователь Chief передал Andrew привилегию UPDATE на таблицу Faculty с правом передавать ее (привилегию) другим пользователям (WITH GRANT OPTION). Воспользовавшись этим правом, Andrew передал пользователю Valery ту же привилегию на Faculty с тем же правом передавать ее другим пользователям.

В связи с этим возникает следующий вопрос: как поступить в этом случае, если у пользователя Andrew забирается право обновлять таблицу Faculty? Для ответа на этот вопрос в язык SQL включены два альтернативных ключевых

слова: CASCADE и RESTRICT. Первое из них означает, что помимо отмены привилегии у Andrew также следует отменить эту же привилегию у Valery и т. д. Второе ключевое слово означает, что нельзя будет отменить привилегию у Andrew, если он уже передал ее другому пользователю. В этом случае необходимо, чтобы сначала Andrew отменил эту привилегию у Valery, и только после этого ее можно будет отменить у Andrew.

Очевидно, если пользователь лишается тех или иных привилегий, он также лишается прав передачи их другим, если они у него имелись. Язык SQL позволяет отменять права предоставления привилегий, сохраняя при этом право пользоваться самими привилегиями. Пусть, например, пользователю Andrew были предоставлены следующие привилегии с правом передачи их другим:

```
GRANT SELECT, UPDATE, DELETE ON Sgroup TO Andrew WITH GRANT OPTION;
```

Теперь, если у него будут **забраны эти привилегии**:

```
REVOKE SELECT, UPDATE, DELETE ON Sgroup FROM Andrew;
```

он также лишается прав передавать их другим. Однако можно следующим образом забрать у Andrew права передачи этих привилегий другим, оставляя возможность ими пользоваться:

```
REVOKE GRANT OPTION FOR SELECT, UPDATE, DELETE ON Sgroup FROM Andrew;
```

После этого у Andrew остаются привилегии SELECT, UPDATE, DELETE по таблице Sgroup, но он уже не может передавать эти привилегии другим.

Можно лишить Andrew прав передачи одних привилегий, оставляя за ним права по другим привилегиям:

```
REVOKE GRANT OPTION FOR UPDATE, DELETE ON Sgroup FROM Andrew;
```

После этого у Andrew все же сохранились права по передаче привилегии SELECT другим пользователям.

Отмена ролей производится следующей командой:

```
REVOKE [ADMIN OPTION FOR] список_ролей FROM {PUBLIC | {пользователь | роль}...} [GRANTED BY {CURRENT_USER | CURRENT_ROLE}] {CASCADE | RESTRICT}
```

Фраза GRANTED BY имеет тот же смысл, что и в предложении GRANT. Роль может быть аннулирована у пользователя или другой роли. При отмене роли у пользователя он теряет те привилегии, которые имеются у этой роли. При отмене роли у другой роли вторая роль теряет те привилегии, которые имеются у первой роли.

При отмене роли у PUBLIC привилегии забираются у тех пользователей, которым они были предоставлены с помощью PUBLIC.

Фраза ADMIN OPTION FOR указывает, что у пользователя забираются права администрирования указанной ролью, однако ее привилегии за ним остаются. Альтернативные ключевые слова CASCADE и RESTRICT указывают, как поступать со ссылочным ограничением целостности, если его существование было разрешено привилегией REFERENCES, которая отменяется предложением REVOKE. В этом случае CASCADE означает, что все такие ссылочные целостности должны быть также удалены. В свою очередь, RESTRICT указывает, что привилегию REFERENCES нельзя

отменить до тех пор, пока согласно ее существованию имеется ссылочная целостность. Далее приведены примеры отмены ролей.

```
REVOKE SpecialRole FROM Smith;  
REVOKE SpecialRole FROM AccountGroupRole;  
REVOKE SpecialRole FROM Public;
```

Привилегии можно определить и используя представления. Каждый раз, когда предоставляются привилегии относительно базовой таблицы, они автоматически распространяются на все строки и столбцы. Если же создать представление, а затем задать привилегии относительно него, то область действия привилегий ограничивается только теми столбцами и строками, которые видны через представление. Это значительно усиливает возможности команды GRANT.

Чтобы иметь возможность выбора данных из представления, необходимо иметь право выбора данных из всех тех базовых таблиц и (или) представлений, на которых данное представление определено. Если представление модифицируемо, любая привилегия INSERT, UPDATE и DELETE, которую имеет пользователь в базовых таблицах, будет автоматически передаваться представлению. Если пользователь не обладает привилегиями на модификацию в базовых таблицах, то не сможет иметь их и в представлениях, которые создал, даже если они модифицируемые.

Так как внешние ключи не определяются относительно представлений, полномочие REFERENCES никогда не используется при создании представления. Предполагается, что владельцы приводимых представлений имеют соответствующие привилегии во всех базовых таблицах.

Для того чтобы дать пользователю John право видеть только столбцы Name, Head, Building таблицы Department, можно создать представление на этих столбцах:

```
CREATE VIEW DepForJohn AS  
SELECT Name, Head, Building FROM Department;
```

и предоставить ему привилегию SELECT относительно этого представления:

```
GRANT SELECT ON DepForJohn TO John;
```

Привилегию SELECT можно указывать в предложении GRANT вместе со списком столбцов, которые разрешается выбирать. Приведенное ранее решение для ограничения доступа по столбцам с использованием представления можно представить и без него следующим образом:

```
GRANT SELECT (Name, Head, Building) ON DepForJohn TO John;
```

Представления оказываются незаменимыми для ограничения действий привилегий на определенные строки базовой таблицы. Для этого определяется представление, содержащее необходимые строки, и относительно него специфицируется привилегия. Например, чтобы **предоставить Andrew привилегию обновления только тех строк таблицы Department, которые относятся к факультету информатики, нужно создать следующее представление:**

```
CREATE VIEW DepFromCS AS  
SELECT * FROM Department
```

```
WHERE FacFK IN (SELECT FacPK FROM Faculty
WHERE LOWER(Name)='информатика') WITH CHECK OPTION;
```

Затем предоставить пользователю Andrew полномочие UPDATE относительно этого представления:

```
GRANT UPDATE ON DepFromCS TO Andrew;
```

Фраза WITH CHECK OPTION в определении представления предохраняет Andrew от изменения столбца FacFK на любое другое значение, которое не является номером факультета информатики.

В некоторых случаях отдельные значения, хранимые в БД, могут носить конфиденциальный характер, и поэтому следует ограничивать к ним доступ. Однако обобщающие данные, базирующиеся на этих значениях, могут быть доступны. С помощью представлений эта проблема разрешима.

Например, если сведения о ставке и надбавке конкретных служащих относятся именно к таким данным, к ним не предоставляются никакие права доступа. Поскольку агрегированные сведения о заработной плате являются обезличенными, их можно делать доступными и в связи с этим определить такое представление:

```
CREATE VIEW AggregateSal SELECT Sum(Salary), SUM(Rise) FROM Teacher;
```

Теперь по отношению к этому представлению можно предоставлять привилегии по чтению, например:

```
GRANT SELECT ON AggregateSal TO PUBLIC;
```

Создание схемы базы данных

Транзакция **CREATE SCHEMA** создает схему в текущей БД. С помощью этой транзакции также можно создавать таблицы и представления в новой схеме и предоставлять разрешения GRANT, DENY или REVOKE на такие объекты.

```
CREATE SCHEMA schema_name_clause [ <schema_element> [ ...n ] ]
<schema_name_clause> ::=
{
  schema_name
| AUTHORIZATION owner_name
| schema_name AUTHORIZATION owner_name
}
<schema_element> ::=
{
  table_definition | view_definition | grant_statement | revoke_statement | deny_statement
}
```

Здесь **Schema_name** — имя, по которому схема идентифицируется в данной БД.

AUTHORIZATION owner_name указывает имя участника уровня БД, который является владельцем схемы. Этому участнику могут принадлежать и другие схемы, при этом текущая схема может не использоваться по умолчанию.

Table_definition указывает инструкцию CREATE TABLE, которая создает таблицу внутри схемы. У участника, выполняющего эту инструкцию, должно быть разрешение CREATE TABLE в текущей БД.

View_definition указывает инструкцию CREATE VIEW, которая создает представление внутри схемы. У участника, выполняющего эту инструкцию, должно быть разрешение CREATE VIEW в текущей БД.

Grant_statement указывает инструкцию GRANT, которая предоставляет разрешения на любой защищаемый объект, за исключением новой схемы.

Revoke_statement указывает инструкцию REVOKE, которая отменяет разрешения на любой защищаемый объект, за исключением новой схемы.

Deny_statement указывает инструкцию DENY, которая запрещает разрешения на любой защищаемый объект, за исключением новой схемы.

Инструкция CREATE SCHEMA создает схему, содержащиеся в ней таблицы и представления, а также разрешения GRANT, REVOKE или DENY на любой защищаемый объект в одной инструкции. Эта инструкция должна выполняться как отдельный пакет. С помощью инструкции CREATE SCHEMA объекты создаются внутри создаваемой схемы.

Транзакции CREATE SCHEMA являются атомарными. Если в процессе выполнения инструкции CREATE SCHEMA возникают ошибки, ни один из указанных защищаемых объектов не создается и ни одно разрешение не предоставляется.

Защищаемые объекты, которые необходимо создать с помощью инструкции CREATE SCHEMA, могут быть перечислены в любом порядке, за исключением представлений, ссылающихся на другие представления. В этом случае ссылающееся представление должно быть создано после того представления, на которое оно ссылается.

Таким образом, с помощью инструкции GRANT можно предоставлять разрешения на объект еще до того, как он будет создан, а инструкция CREATE VIEW может появляться раньше инструкций CREATE TABLE, создающих таблицы, на которые ссылается представление. Кроме того, инструкции CREATE TABLE могут декларировать внешние ключи к таблицам, определенным позже в инструкции CREATE SCHEMA.

В инструкциях CREATE SCHEMA поддерживаются DENY и REVOKE. Предложения DENY и REVOKE будут исполняться в той последовательности, в которой они появляются в инструкции CREATE SCHEMA.

Участник, выполняющий инструкцию CREATE SCHEMA, может указать другого участника БД в качестве владельца создаваемой схемы. Для этого необходимы дополнительные разрешения, описанные далее.

Новая схема принадлежит одному из следующих участников уровня БД: пользователю БД, роли БД или роли приложения. Владение объектами, содержащимися в схеме, можно передать любому участнику уровня БД, однако у владельца схемы всегда остается разрешение CONTROL на объекты в схеме.

Для того чтобы создать объект, указанный в инструкции CREATE SCHEMA, у пользователя должно быть соответствующее разрешение CREATE.

Чтобы назначить другого пользователя владельцем создаваемой схемы, у участника должно быть разрешение IMPERSONATE на этого пользователя. Если роль БД указана в качестве владельца, то вызывающий объект должен входить в роль или иметь на нее разрешение ALTER.

В следующем примере создается схема Sprockets, принадлежащая Annik, которая содержит таблицу NineProngs. Инструкция предоставляет разрешение SELECT для John и запрещает SELECT для Smith.

Схема, таблица и разрешения для разных пользователей создаются в одной инструкции.

```
USE AdventureWorks2024;  
GO  
CREATE SCHEMA Sprockets AUTHORIZATION Andrew;  
CREATE TABLE NineProngs (source int, cost int, partnumber int)  
GRANT SELECT ON SCHEMA::Sprockets TO John;  
DENY SELECT ON SCHEMA::Sprockets TO Smith;  
GO
```